Phase 2: Implementing the LPN-1 Microprocessor in gem5

Nithin Charan Boppana

Computer Architecture and Design (MSCS-531-M40)

11/07/2026

Phase 2: Implementing the LPN-1 Microprocessor in gem5

This guide explains how I built LPN-1, the low-power sensor-node processor defined in Phase 1, in gem5, and how I checked it. All steps ran on a MacBook Air with gem5 v25.1.0.1. Energy figures are model estimates based on assumed coefficients, not measurements of a silicon chip.

Installing and configuring gem5

First, I built the RISC-V target from the gem5 source with scons build/RISCV/gem5.opt -j2, using caffeinate to keep the Mac awake. Second, I compiled programs in a Docker container (Debian Trixie) using gcc-riscv64-linux-gnu, thereby avoiding the need to install a cross-compiler on macOS. Third, I checked that the source supports what LPN-1 needs. The RISC-V ISA has a riscv_type setting (RV32 or RV64), the user-mode process code has an RV32 class, and the in-order Minor CPU and the power-model classes are present. gem5's standard library builds a system from Python objects (board, processor, caches, memory), which is how my configuration script is written (Lowe-Power et al., 2020). The project lives in the folder lpn1/.

Implementing LPN-1

Step 1 Workloads: I wrote four sensor-node kernels in C: a 16-tap FIR filter, feature extraction, a CRC-16 checksum, and a full pipeline that chains them. They use no C library, so each program has its own entry point and uses system calls to print a checksum and exit. I built each kernel twice: once as RV32IMC and once as RV32IM without compressed instructions. Figure 1 shows the ELF header and the code sizes. The compressed build has 782 bytes of code and the uncompressed build 1,016 bytes, which is 23% smaller.

Step 2 Core: The script lpn1_run.py creates a Minor CPU, sets riscv_type to RV32, and limits the decode, issue, and commit widths to 1.

Step 3 Caches: both L1 caches are 8KiB and 2-way. The standard library didn't allow me to set the associativity, so I've made a subclass that sets it to 2 after the caches have been created.

Step 4 Memory: A 128 KiB memory with 10 ns latency is the on-chip SRAM.

Step 5 DVFS: Each run sets the clock frequency and voltage domain, so each run represents one operating point.

Step 6 Power: I attached a power model to the core with the four gem5 states (ON, CLK_GATED, SRAM_RETENTION, OFF), only the ON state being non-zero.

A second script (energy.py) adds in cache, memory and leakage energy from the run stats. The coefficients are all assumptions: 10 pJ per cache access at 1V is the order of magnitude given in Horowitz (2014) and the retention leakage factor of 0.575 is from Machetti et al. (2024). Table 1 shows the differences between the model and the Phase 1 design.

Figure 1

Build Output for the Pipeline Kernel

```
nithinboppana@Nithins-MacBook-Air gem5 % ./lpn1/scripts/build_kernels.sh 2>&1 | tail -12
--- ELF header and code size of the pipeline kernel (C extension on / off) ---
  Class:                             ELF32
  Machine:                           RISC-V
  Flags:                             0x1, RVC, soft-float ABI
   text    data     bss     dec     hex filename
    782       4    1056    1842     732 bin/node_pipe_c
   1016       4    1056    2076     81c bin/node_pipe_nc
Host reference checksums:
checksum 04b4fd49
checksum a6c4e687
checksum b7b8cf3e
checksum 6e5cc518
```

Note. The header shows a 32-bit RISC-V file with the compressed (RVC) flag. The last four lines are the checksums computed on the host.

Table 1
LPN-1 Design Compared With the gem5 Model

Feature	Phase 1 design	gem5 model	Difference
Instruction set	RV32IMC	RV32 core; programs built with and without C	Model also enables the vector extension (unused)
Pipeline	Short, in-order, single issue	Minor CPU, widths set to 1	Stage structure is fixed by Minor CPU
Branch prediction	Static, no BTB	gem5 default predictor	Model has a BTB
L1 caches	8 KiB I and D, 2-way	Same	None
Memory	128 KiB, four banks	One 128 KiB memory, 10 ns	Banks are not modeled
DVFS	Three operating points	One point per run	No switching inside a run
Gating	Clock gating, retention, power off	Core power model with four states	Sleep states not simulated over time

Note. BTB = branch-target buffer.

Validation and Preliminary Results

I checked it in three ways. Firstly, for correctness the gem5 checksum of each program must match the host checksum. Secondly for configuration the saved config must show the correct values (see Figure 2). And finally for behavior the DVFS, cache size and compression runs must all behave as expected.

Table 2
Checksum Validation of the Four Kernels

Program	Host reference	gem5, compressed	gem5, uncompressed	Match
FIR	04b4fd49	04b4fd49	04b4fd49	Yes
Feature	a6c4e687	a6c4e687	a6c4e687	Yes
CRC-16	b7b8cf3e	b7b8cf3e	b7b8cf3e	Yes
Pipeline	6e5cc518	6e5cc518	6e5cc518	Yes

Figure 2
Configuration and Power-Model Evidence

```
$ grep -E 'riscv_type|clock=|voltage=' m5out/lpn1_pipe_c/config.ini
clock=20000
voltage=0.8
riscv_type=RV32

[board.cache_hierarchy.l1dcaches]    (config.ini excerpt)
assoc=2
size=8192

$ grep -E 'power_model\.(dynamic|static)Power' m5out/lpn1_pipe_c_pm/stats.txt
board...core.power_model.dynamicPower     0.000446     # Watt
board...core.power_model.staticPower      0.000016     # Watt
```

Note. Excerpts from the saved config.ini and stats.txt of the pipeline run. The clock value 20000 is a period in picoseconds, which is 50 MHz.

All eight programs matched (Table 2), so gem5 decodes both builds correctly and 128 KiB of memory is enough. The statistics also confirm 50 MHz (2,381,086 cycles in 47.622 ms).

Table 3
DVFS Sweep of the Pipeline Kernel (Model Estimates)

Operating point	Cycles	Time (ms)	Dynamic (µJ)	Leakage (µJ)	Total (µJ)	pJ per instruction
1.0 V / 100 MHz	2,381,086	23.8	43.8	1.7	45.5	32.1
0.8 V / 50 MHz	2,381,086	47.6	28.0	3.1	31.1	21.9
0.6 V / 20 MHz	2,381,086	119.1	15.8	6.9	22.7	16.0

Note. Dynamic energy includes the core and both L1 caches; memory energy is under 0.01 µJ.

The cycle count is the same at all three points and execution time scales with 1/f (Table 3). In the model dynamic energy scales with the square of the voltage (100%, 64% and 36% of peak for the core respectively), whilst leakage energy increases as the run progresses. Total active energy is lowest at 0.6V and 20MHz, though the run takes five times longer than at 100MHz. Memory leakage is the dominant leakage term in this model (40 of 58µW at 0.6V) and hence supports the gating of unused banks. The best point also depends on sleep energy which Phase 3 will look at.

Table 4
Effect of Compressed Instructions on the Four Kernels

Program	Cycles (C)	Cycles (no C)	Time change	L1I acc. (C)	L1I acc. (no C)	Acc. change
FIR	1,188,876	1,029,885	+15.4%	385,142	308,377	+24.9%
Feature	227,904	222,529	+2.4%	63,802	63,482	+0.5%
CRC-16	1,334,643	1,340,047	−0.4%	551,688	469,865	+17.4%
Pipeline	2,381,086	2,383,446	−0.1%	868,743	841,983	+3.2%

Note. C = compressed instructions; L1I acc. = L1 instruction-cache accesses.

Two other checks worked as expected. With 2KiB L1 caches we saw data cache misses double from 13 to 26 and cycles only increase by 31, so the cache setting is in fact taking effect, and these kernels do fit in a small cache. The core power model reports 446uW dynamic and 16uW static. 446uW x run time = 21.2uJ which is the same as the core dynamic energy from energy.py. That's consistent with the implementation, though again it's using the same coefficients, so it doesn't validate them.

The compressed instruction result is different from what I'd anticipated. The code is 23% smaller with compression, but it hasn't reduced instruction cache accesses in gem5 at all. They've increased by 0.5% to 25%, and the FIR kernel is now running 15% longer (Table 4). Waterman (2011) reports about 25% fewer instruction bits fetched, though gem5 counts fetches rather than bits. I haven't yet established the cause of this, that will be the subject of Phase 3 which will look at the fetch stats.

Issues Encountered and Fixes

Two issues arose. Firstly, gem5 issued a warning that it couldn't set the cache associativity and defaulted to its own value. That was fixed in step 3 and assoc=2 is now shown in config.ini (Figure 2). Secondly the first power model run gave a dynamic power of −290,753,858 W and a static power of 15.2 W. gem5's expression parser doesn't accept scientific notation, so it took 20e-6 * voltage as $20 - 6 \times 0.8 = 15.2$. Writing the numbers out as plain decimals gave the values above.

References

gem5 Project. (n.d.). *gem5 documentation*.
https://www.gem5.org/documentation/

Horowitz, M. (2014) 'Computing's energy problem (and what we can do about it)' in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (pp.*10–14). IEEE.
https://doi.org/10.1109/ISSCC.2014.6757323

Lowe-Power, J., Ahmad, A. M., Akram, A., Alian, M., Amslinger, R., Andreozzi, M., Armejach, A., Asmussen, N., Beckmann, B., Bharadwaj, S., Black, G., Bloom, G., Bruce, B. R., Carvalho, D. R., Castrillon, J., Chen, L., Derumigny, N., Diestelhorst, S., Elsasser, W., ... Zulian, E. (2020). *The gem5 simulator: Version 20.0+* (arXiv:2007.03152). arXiv.
https://arxiv.org/abs/2007.03152

Machetti, S., Schiavone, P. D., Müller, T. C., Peón-Quirós, M. & Atienza, D. (2024) *X-HEEP: An open-source, configurable and extendible RISC-V microcontroller for the exploration of ultra-low-power edge accelerators* (arXiv:2401.05548). arXiv.
https://arxiv.org/abs/2401.05548

Waterman, A. (2011). *Improving energy efficiency and reducing code size with RISC-V compressed* (Technical Report No. UCB/EECS-2011-63). University of California, Berkeley.
https://www2.eecs.berkeley.edu/Pubs/TechRpts/2011/EECS-2011-63.html